Rapport Projet POO – Version 2 (POO-v2)

1. Fonctionnalités développées dans cette version

Dans cette deuxième version du projet, nous avons poursuivi le développement de notre outil ,en intégrant deux éléments majeurs:

1. **La gestion de la validité des critères via un mécanisme d'exception.**
2. **L'implémentation de règles de compatibilité spécifiques à certains pays, notamment la France.**

Nous avons également entrepris un refactoring important de notre structure de données, afin de mieux respecter les consignes de l'énoncé.

🔧 Ce qui a changé :

- La map initiale **`Map<Critere,Object>`** a été remplacée par une **structure plus propre** :

```
Map<Critere, String>
```

🎯 Pourquoi ce changement ?

- L'énoncé indique explicitement que **les données proviennent de fichiers CSV**, et que **toutes les valeurs y sont textuelles**.
- L'utilisation de **Object** était trop vague, difficile à maintenir, et obligeait à faire des conversions inutiles (`instanceof`, cast, etc.).
- Avec des **`String`**, le code est plus simple, plus sûr, plus lisible et plus cohérent avec le fichier source.

1) Gestion de la validité des critères (mécanisme d'exception)

Nous avons mis en place une validation automatique des critères renseignés par les adolescents à travers une méthode **`validerCritere(Critere, String)`**.

Cette méthode :

- Vérifie que les critères de type booléen (`'B'`) contiennent bien uniquement `"yes"` ou `"no"`.
- Vérifie que les critères de type texte (`'T'`) ne sont pas vides.
- Provoque une exception personnalisée (**`CritereInvalideException`**) en cas de valeur incohérente.

2) Règles de compatibilité spécifiques à certains pays

🔧 Fonctionnalité ajoutée :

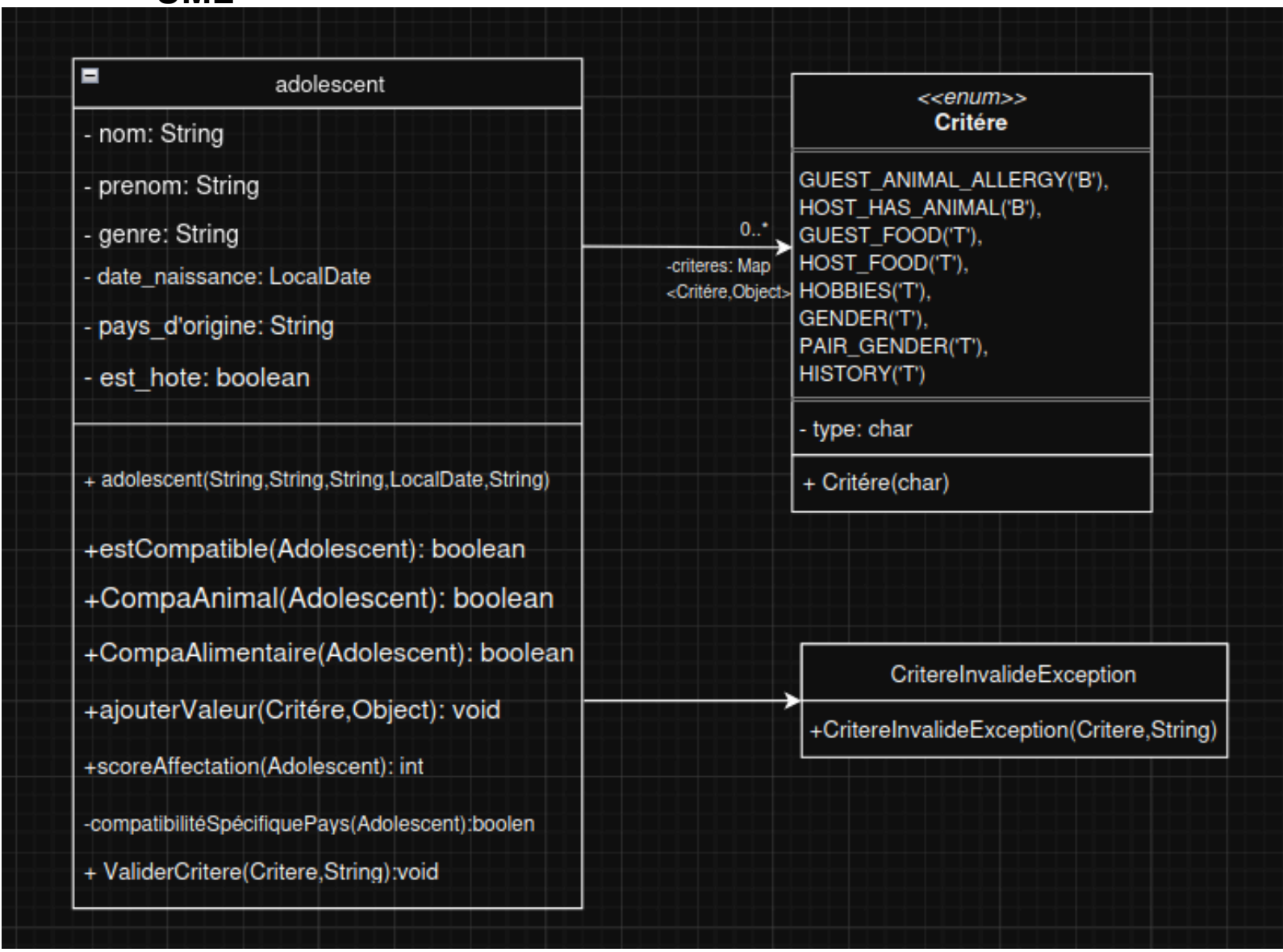
Conformément à l'énoncé, nous avons modifié la méthode `estCompatible(Adolescent autre)` pour inclure une règle spécifique aux adolescents français.

> Si l'un des deux adolescents vient de France, ils doivent partager au moins un passe-temps (critère **HOBBIES**) pour être considérés comme compatibles**.**

📘 Implémentation :

- Ajout d'une méthode `compatibiliteSpecifiquePays(...).`
- Ajout de cette condition dans la compatibilité globale.

UML


adolescent
- nom: String
- prenom: String
- genre: String
- date_naissance: LocalDate
- pays_d'origine: String
- est_hote: boolean
+ adolescent(String,String,String,LocalDate,String)
+estCompatible(Adolescent): boolean
+CompaAnimal(Adolescent): boolean
+CompaAlimentaire(Adolescent): boolean
+ajouterValeur(Critére,Object): void
+scoreAffectation(Adolescent): int
-compatibilitéSpécifiquePays(Adolescent):boolen
+ ValiderCritere(Critere,String):void
0..*
-criteres: Map
<Critére,Object>
<<enum>>
Critére
GUEST_ANIMAL_ALLERGY('B'),
HOST_HAS_ANIMAL('B'),
GUEST_FOOD('T'),
HOST_FOOD('T'),
HOBBIES('T'),
GENDER('T'),
PAIR_GENDER('T'),
HISTORY('T')
- type: char
+ Critére(char)
CritereInvalideException
+CritereInvalideException(Critere,String)